BST

Binary Search Tree (BST) Basics

A **Binary Search Tree (BST)** is a specific type of binary tree where the nodes are ordered according to the following rules for every node X:

1. All values in the left subtree of X must be less than X's value.
2. All values in the right subtree of X must be greater than X's value.

This ordering principle enables efficient searching.

Time Complexity of BST Operations

The complexity of BST operations depends heavily on the tree's height (h).

Operation	Average Case (Balanced Tree)	Worst Case (Skewed Tree)
Search	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$

- **Average $O(\log n)$:** Achieved when the tree is **balanced**, allowing the search space to be halved at every step.
- **Worst Case $O(n)$:** Occurs when the tree is **skewed** (like a linked list), forcing a linear traversal.

Finding Successor and Predecessor After Deletion

When a node with two children is deleted, it must be replaced by another node to maintain the BST property.

Both the Successor and Predecessor are valid replacements.

The successor of a node X in a BST is the smallest key in X's right subtree

The predecessor is the largest key in X's left subtree.

1. Successor (The Next Largest Value) ➡

The successor is the node with the **smallest key in the right subtree** of the node being deleted.

- **How to Find:** Go **one step to the right child**, then travel **all the way down to the left** until you find a node with no left child.
- **Use in Deletion:** The successor is cut from its original position (where it had at most one child) and moved to the deleted node's position.

2. Predecessor (The Next Smallest Value) ⬅

The predecessor is the node with the **largest key in the left subtree** of the node being deleted.

- **How to Find:** Go **one step to the left child**, then travel **all the way down to the right** until you find a node with no right child.
- **Use in Deletion:** It serves as an equally valid replacement for the deleted node as the successor.